# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Fundamentals of Data Science** **COURSE CODE: DSA0401**

---

**COURSE OUTCOME COVERED**

***CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)***

*Assessment Tool Weightage: 30%*

---

**QUESTIONS: 10** **Total Marks:100**

Annexure A

# Scenario-Based Assessment

---

***Code of Conduct:***

*I* ***A.Sai Rohit (192472144)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: A.Sai Rohit***

**RUBRICS FOR CALCULATION**

| SCENARIO BASED ASSESSMENT | | | | | | | |
|---|---|---|---|---|---|---|---|
| **Criteria** | **Weightage (%)** | **Level 4 (Exemplary)** | **Level 3 (Proficient)** | **Level 2 (Developing)** | **Level 1 (Beginning)** | Level | Marks |
| **Understanding of Scenario** | 20% | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps | Partial understanding; misses key aspects | Misinterprets or fails to understand the scenario | | |
| **Identification of Key Issues** | 20% | Accurately identifies all relevant issues and factors involved | Identifies most key issues with minor omissions | Identifies limited issues; some irrelevant points | Unable to identify key issues | | |
| **Application of Concepts** | 25% | Applies appropriate concepts effectively to address the scenario | Applies concepts with minor errors | Limited or partially correct application | Unable to apply relevant concepts | | |
| **Decision Making** | 20% | Makes well-reasoned, logical decisions with strong rationale | Makes reasonable decisions with some justification | Makes weak decisions with limited justification | No clear decisions made or rationale provided | | |
| **Clarity of Response** | 15% | Response is clear, structured, and logically presented | Generally clear with minor issues | Somewhat unclear or poorly structured | Disorganized and difficult to understand | | |

# Annexure A

**NOTE:**

For every question, answer the following:

a) Identify the numerical and binary features used for KNN classification.
b) Calculate the **Euclidean distance** between the new sample and all training samples
c) Calculate the **Manhattan distance** between the new sample and all training samples.
d) Calculate the **Minkowski distance** with **p = 3** between the new sample and all training samples.
e) Calculate the **Hamming distance** between the new sample and all training samples using binary features.
f) Identify the **3 nearest neighbours** separately for each distance measure.
g) Predict the class label using majority voting for each distance measure.
h) Compare the results and explain which distance measure is most suitable for the given scenario.

**Total = 100 Marks**

**Question 1: Student Pass or Fail Prediction**

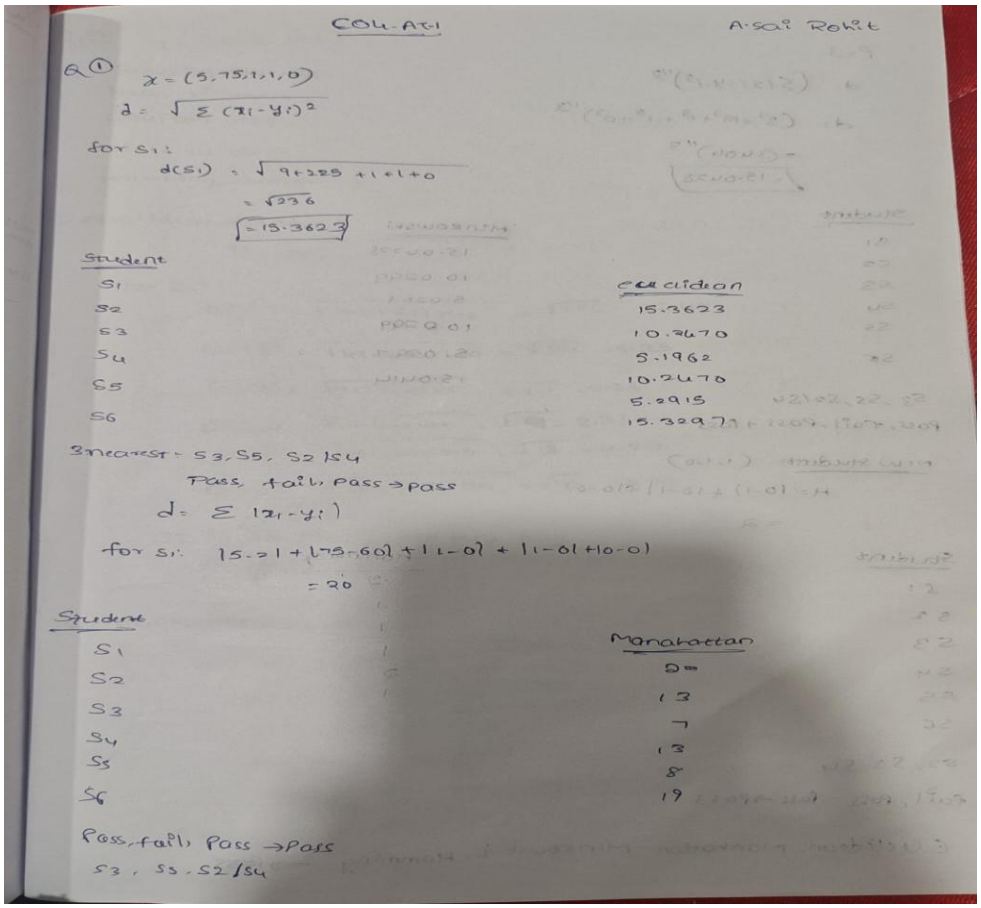
A college wants to predict whether a student will Pass or Fail based on academic and activity-related details.

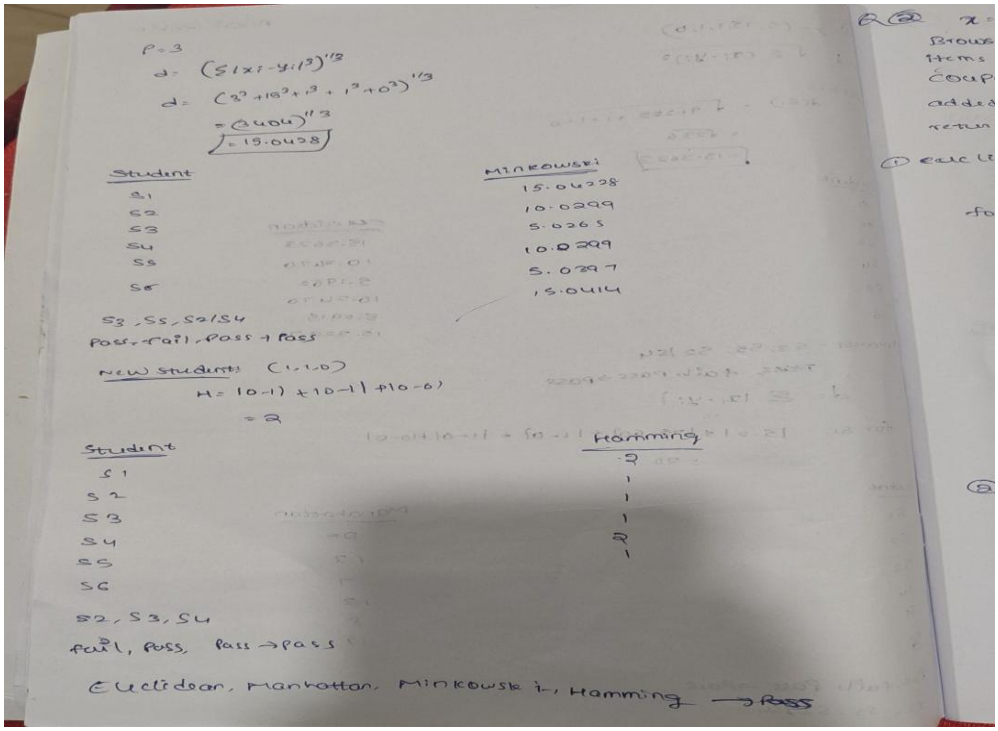
| Student | Study Hours | Attendance % | Assignment Submitted | Lab Participation | Revision Done | Result |
|---|---|---|---|---|---|---|
| S1 | 2 | 60 | 0 | 0 | 0 | Fail |
| S2 | 3 | 65 | 1 | 0 | 0 | Fail |
| S3 | 6 | 80 | 1 | 1 | 1 | Pass |
| S4 | 7 | 85 | 1 | 1 | 1 | Pass |
| S5 | 4 | 70 | 1 | 0 | 1 | Fail |
| S6 | 8 | 90 | 1 | 1 | 1 | Pass |

A new student has the following details:

| Study Hours | Attendance % | Assignment Submitted | Lab Participation | Revision Done |
|---|---|---|---|---|
| 5 | 75 | 1 | 1 | 0 |

Using KNN with K = 3, classify the new student as Pass or Fail using Euclidean, Manhattan, Minkowski, and Hamming distance.






**Code:**

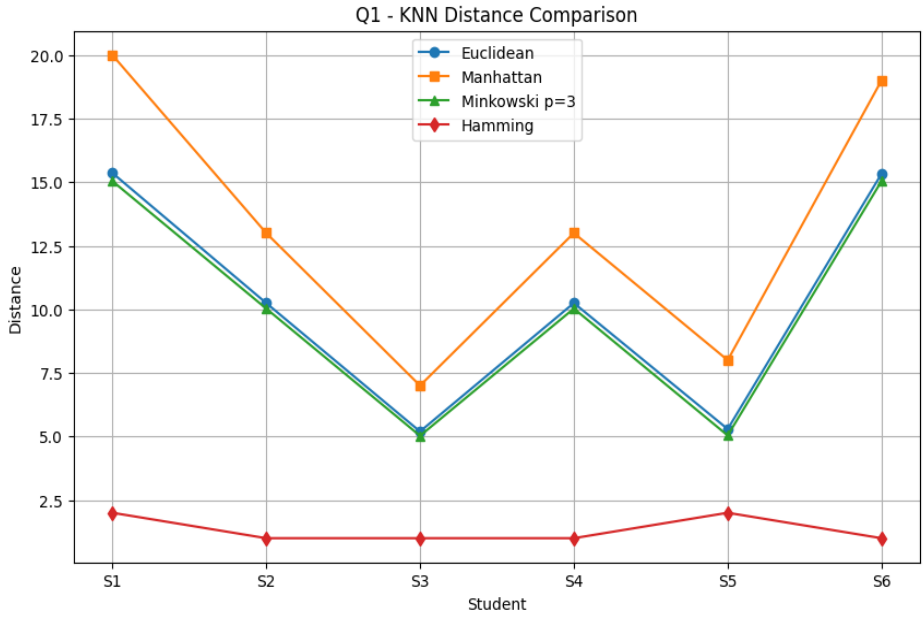
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

from google.colab import files


uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

features = ["Height", "Weight", "Age", "Exercise_Hours", "Sleep_Hours"]

new_sample = np.array([170, 65, 21, 2, 7])
```

```
X = df[features].astype(float).values
euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)

binary_features = ["Smoker", "Alcohol", "Gym"]
hamming = np.sum(
    df[binary_features].astype(int).values != np.array([0, 0, 1]),
    axis=1
)
result = pd.DataFrame({
    "Person": df["Person"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})
display(result)
for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
    print(method)
    print("Nearest:", list(nearest["Person"]))
    print("Prediction:", prediction)
```

```
plt.figure(figsize=(10, 6))
plt.plot(result["Person"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["Person"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["Person"], result["Minkowski"], marker="^", label="Minkowski")
plt.plot(result["Person"], result["Hamming"], marker="d", label="Hamming")
plt.xlabel("Person")
plt.ylabel("Distance")
plt.title("Q1 - KNN Distance Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

**Output:**

```
============================================================
FINAL PREDICTION
============================================================
Euclidean: Pass
Manhattan: Pass
Minkowski_p3: Pass
Hamming: Pass
```



**Question 2: Online Customer Purchase Prediction**

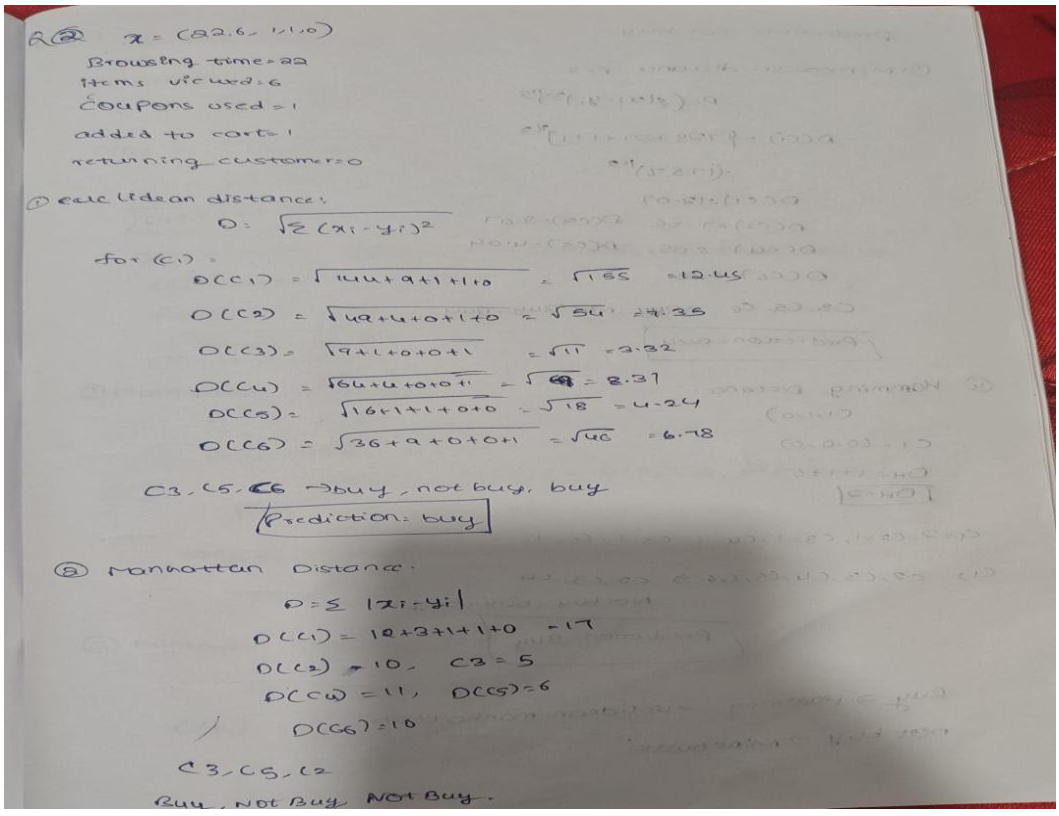
An e-commerce company wants to predict whether a customer will Buy or Not Buy a product.

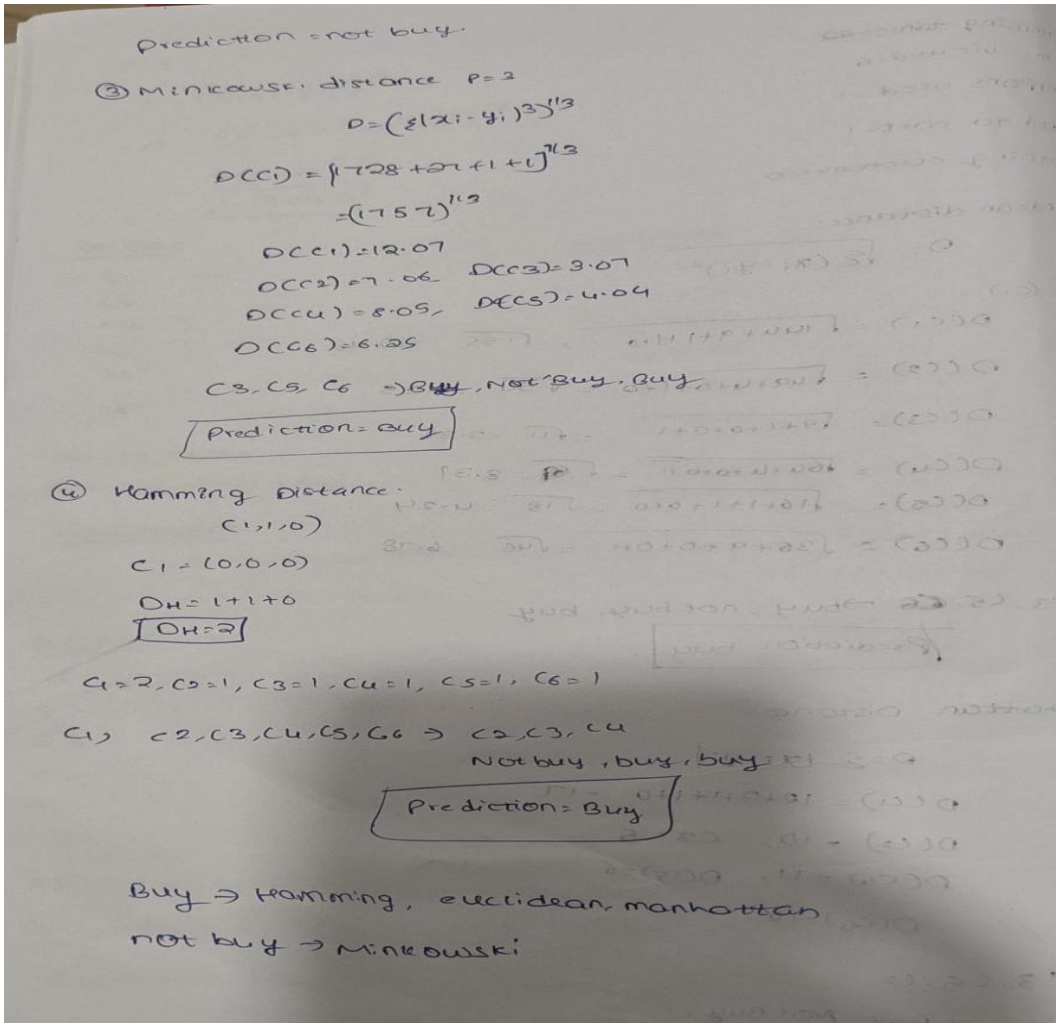
| Customer | Browsing Time | Items Viewed | Coupon Used | Added to Cart | Returning Customer | Class |
|---|---|---|---|---|---|---|
| C1 | 10 | 3 | 0 | 0 | 0 | Not Buy |
| C2 | 15 | 4 | 1 | 0 | 0 | Not Buy |
| C3 | 25 | 7 | 1 | 1 | 1 | Buy |
| C4 | 30 | 8 | 1 | 1 | 1 | Buy |
| C5 | 18 | 5 | 0 | 1 | 0 | Not Buy |
| C6 | 28 | 9 | 1 | 1 | 1 | Buy |

A new customer has:

| Browsing Time | Items Viewed | Coupon Used | Added to Cart | Returning Customer |
|---|---|---|---|---|
| 22 | 6 | 1 | 1 | 0 |

Using KNN with K = 3, classify the new customer as Buy or Not Buy using all four distance measures.






**Code:**

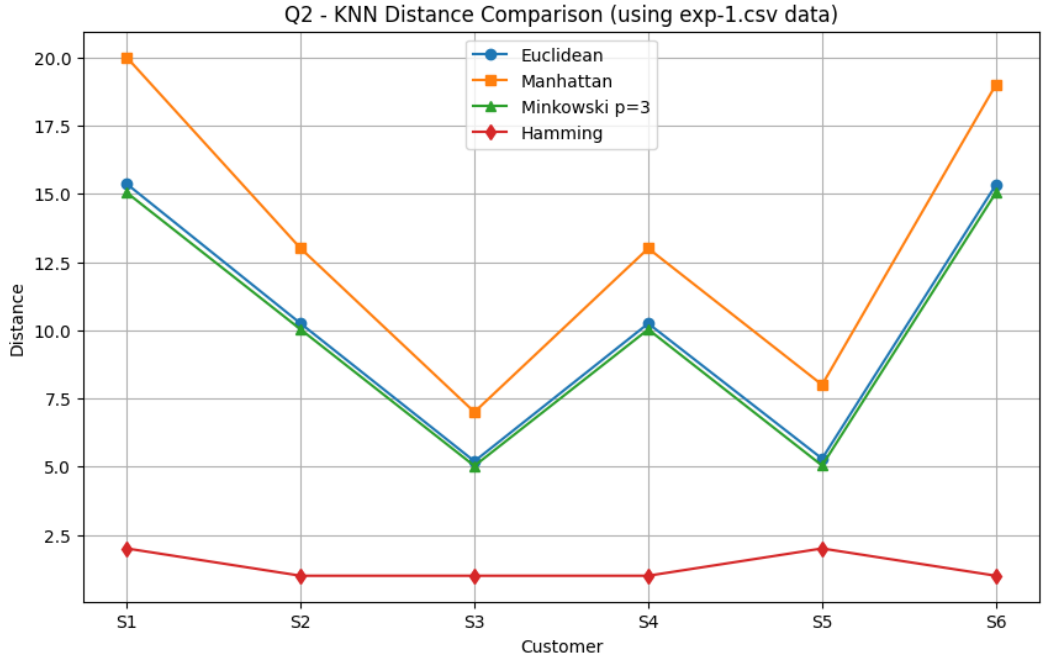
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

from google.colab import files

uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)


features = ["Age", "Income", "Credit_Score", "Experience", "Loan_Amount"]
```

```
new_sample = np.array([30, 50000, 700, 5, 200000])
X = df[features].astype(float).values
euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)
binary_features = ["Married", "Home_Owner", "Previous_Loan"]
hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 1, 0]),
    axis=1
)
result = pd.DataFrame({
    "Applicant": df["Applicant"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})
display(result)
for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
    print(method)
    print("Nearest:", list(nearest["Applicant"]))
    print("Prediction:", prediction)
```

```
plt.figure(figsize=(10, 6))
plt.plot(result["Applicant"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["Applicant"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["Applicant"], result["Minkowski"], marker="^", label="Minkowski")
plt.plot(result["Applicant"], result["Hamming"], marker="d", label="Hamming")
plt.xlabel("Applicant")
plt.ylabel("Distance")
plt.title("Q2 - KNN Distance Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

**Output:**

```
============================================================
FINAL PREDICTION
============================================================
Euclidean: Pass
Manhattan: Pass
Minkowski_p3: Pass
Hamming: Pass
```



**Question 3: Medical Risk Classification**

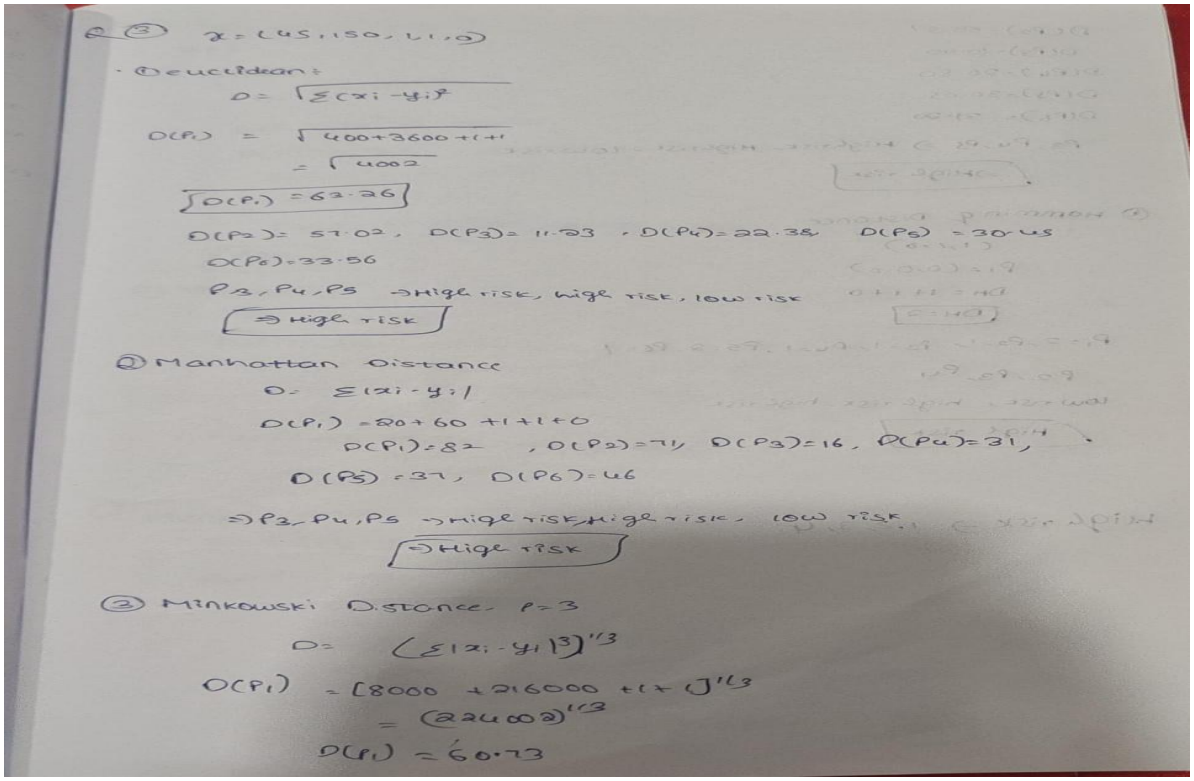
A hospital wants to classify whether a patient has High Risk or Low Risk based on medical measurements and symptoms.

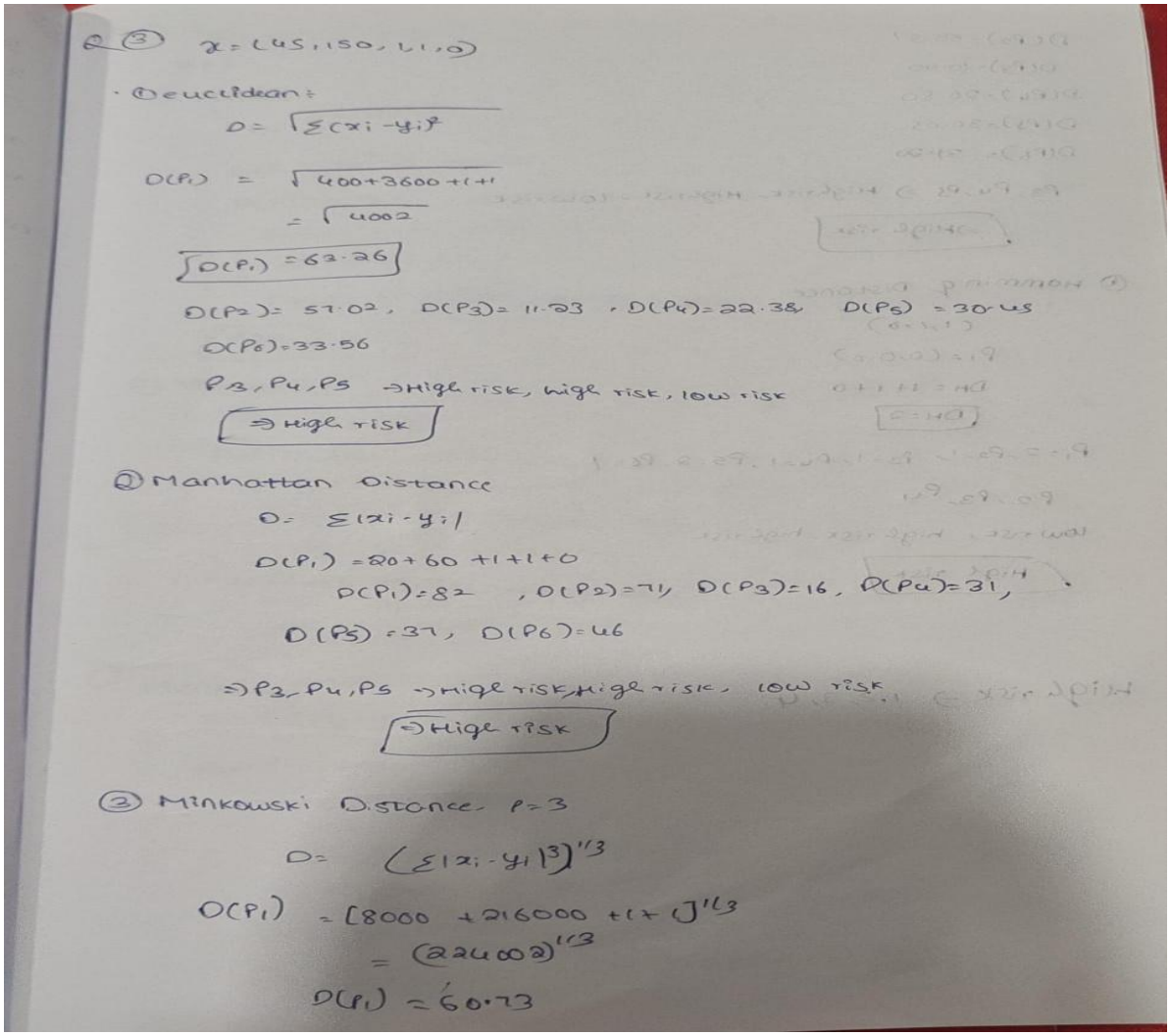
| Patient | Age | Blood Sugar Level | Fever | Cough | Fatigue | Class |
|---|---|---|---|---|---|---|
| P1 | 25 | 90 | 0 | 0 | 0 | Low Risk |
| P2 | 30 | 95 | 0 | 1 | 0 | Low Risk |
| P3 | 50 | 160 | 1 | 1 | 1 | High Risk |
| P4 | 55 | 170 | 1 | 1 | 1 | High Risk |
| P5 | 40 | 120 | 0 | 1 | 1 | Low Risk |
| P6 | 60 | 180 | 1 | 1 | 1 | High Risk |

A new patient has:

| Age | Blood Sugar Level | Fever | Cough | Fatigue |
|---|---|---|---|---|
| 45 | 150 | 1 | 1 | 0 |

Using KNN with K = 3, classify the patient as High Risk or Low Risk using Euclidean, Manhattan, Minkowski, and Hamming distance.






**Code:**

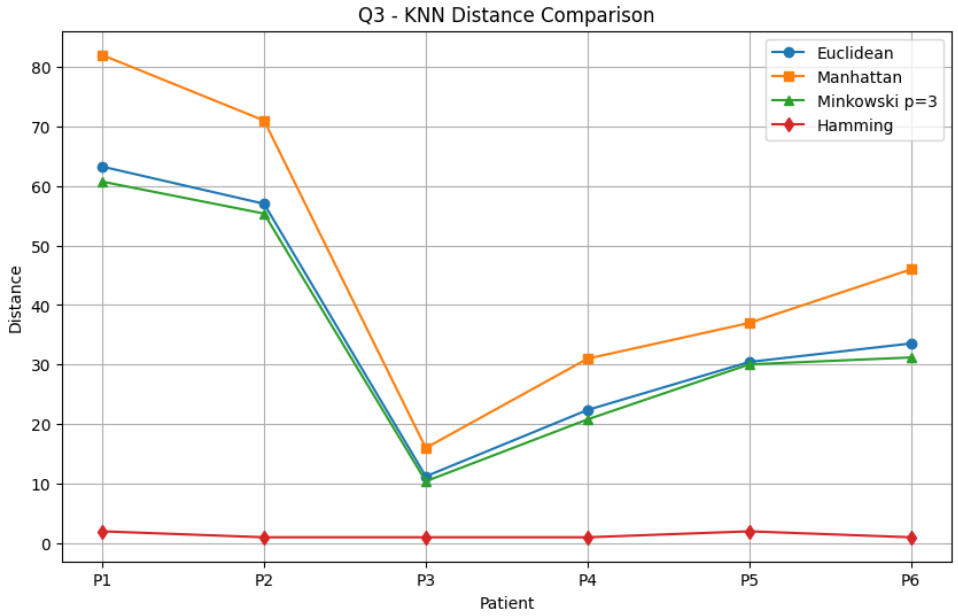
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

from google.colab import files


uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)
```

```
features = ["Speed", "Agility", "Strength", "Endurance", "Training_Hours"]

new_sample = np.array([80, 75, 70, 85, 6])

X = df[features].astype(float).values

euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)

binary_features = ["Team_Player", "Injured", "Captain"]

hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 0, 0]),
    axis=1
)

result = pd.DataFrame({
    "Player": df["Player"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
```

```
})

display(result)

for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
    print(method)
    print("Nearest:", list(nearest["Player"]))
    print("Prediction:", prediction)

plt.figure(figsize=(10, 6))
plt.plot(result["Player"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["Player"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["Player"], result["Minkowski"], marker="^", label="Minkowski")
plt.plot(result["Player"], result["Hamming"], marker="d", label="Hamming")
plt.xlabel("Player")
plt.ylabel("Distance")
plt.title("Q3 - KNN Distance Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

**Output:**

```
============================================================
FINAL PREDICTION
============================================================
Euclidean: High Risk
Manhattan: High Risk
Minkowski_p3: High Risk
Hamming: High Risk
```



**Question 4: Fruit Classification**

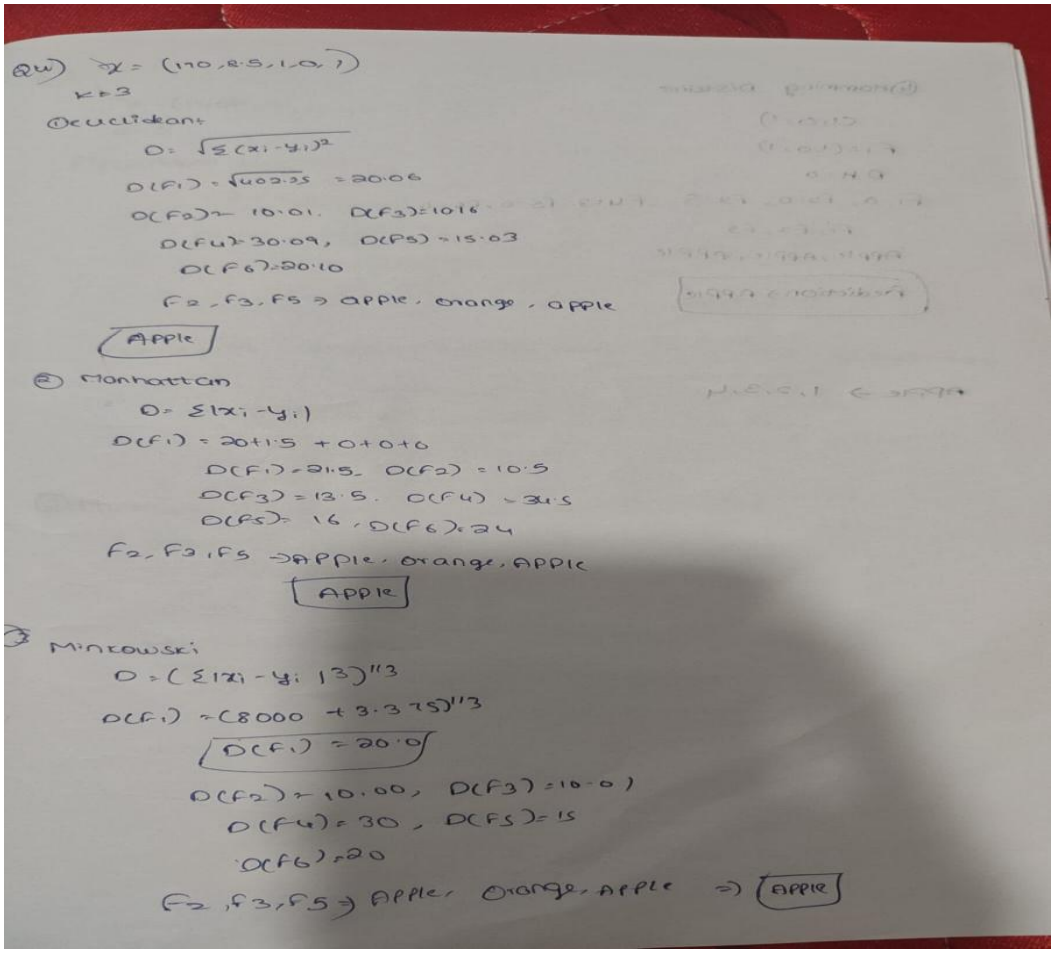
A fruit shop wants to classify a fruit as Apple or Orange based on physical and binary characteristics.

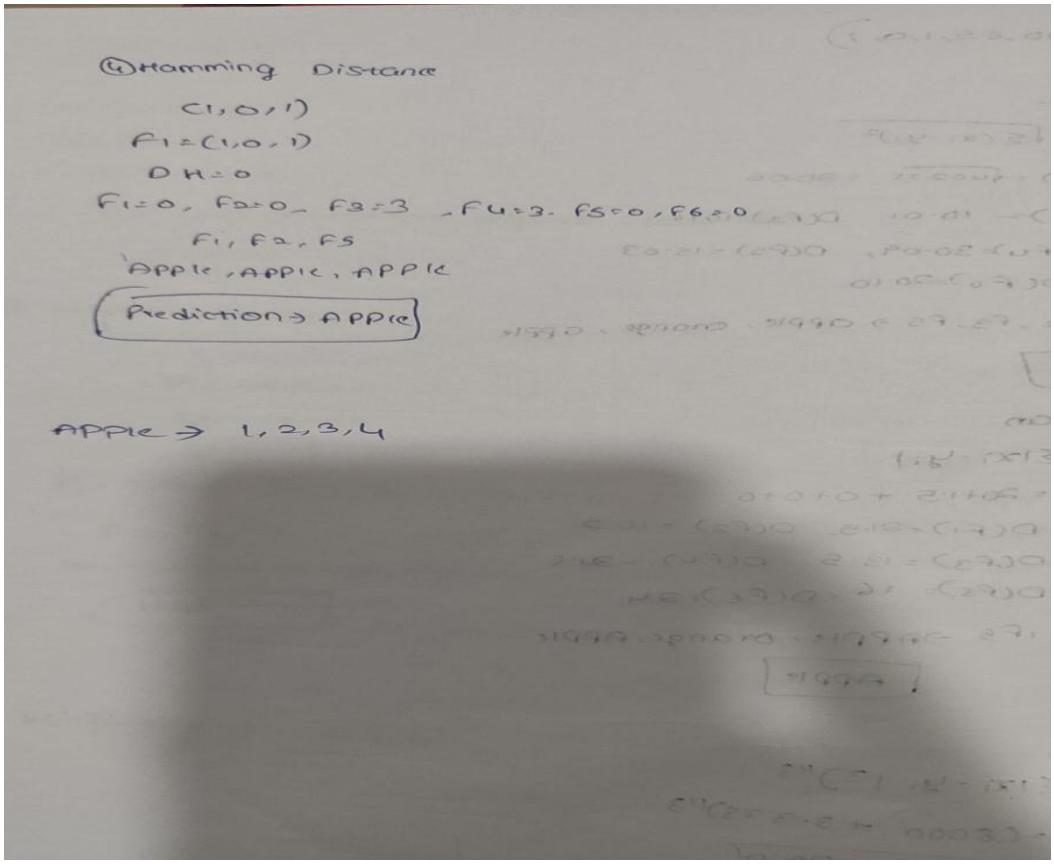
| Fruit | Weight | Diameter | Red Color | Citrus Smell | Smooth Skin | Class |
|---|---|---|---|---|---|---|
| F1 | 150 | 7 | 1 | 0 | 1 | Apple |
| F2 | 160 | 8 | 1 | 0 | 1 | Apple |
| F3 | 180 | 9 | 0 | 1 | 0 | Orange |
| F4 | 200 | 10 | 0 | 1 | 0 | Orange |
| F5 | 155 | 7.5 | 1 | 0 | 1 | Apple |
| F6 | 190 | 9.5 | 0 | 1 | 0 | Orange |

A new fruit has:

| Weight | Diameter | Red Color | Citrus Smell | Smooth Skin |
|---|---|---|---|---|
| 170 | 8.5 | 1 | 0 | 1 |

Using KNN with K = 3, classify the fruit as Apple or Orange using all four distance measures.






**Code:**

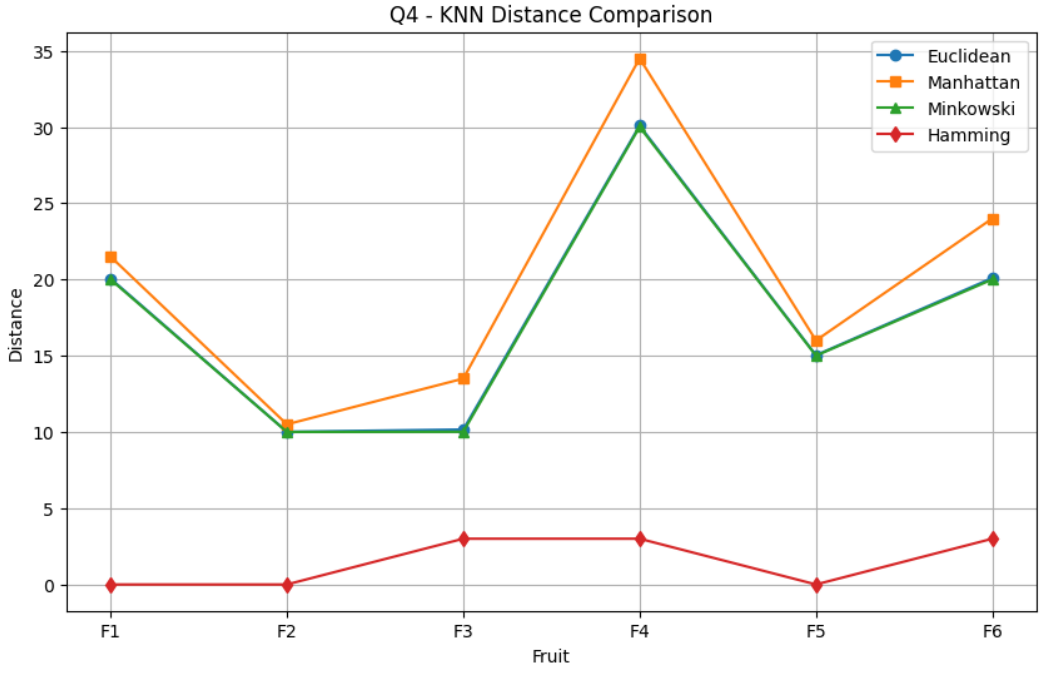
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

from google.colab import files

uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

features = ["Weight", "Diameter", "Red_Color", "Citrus_Smell", "Smooth_Skin"]

new_sample = np.array([170, 8.5, 1, 0, 1])

X = df[features].astype(float).values

euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
```

```
manhattan = np.sum(np.abs(X - new_sample), axis=1)
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)
binary_features = ["Red_Color", "Citrus_Smell", "Smooth_Skin"]
hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 0, 1]),
    axis=1
)
result = pd.DataFrame({
    "Fruit": df["Fruit"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})
display(result)
for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
    print(method)
    print("Nearest:", list(nearest["Fruit"]))
    print("Prediction:", prediction)
plt.figure(figsize=(10, 6))
plt.plot(result["Fruit"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["Fruit"], result["Manhattan"], marker="s", label="Manhattan")
```

```
plt.plot(result["Fruit"], result["Minkowski"], marker="^", label="Minkowski")

plt.plot(result["Fruit"], result["Hamming"], marker="d", label="Hamming")

plt.xlabel("Fruit")

plt.ylabel("Distance")

plt.title("Q4 - KNN Distance Comparison")

plt.legend()

plt.grid(True)

plt.show()
```

**Output:**

```
Euclidean
Nearest: ['F2', 'F3', 'F5']
Prediction: Apple
Manhattan
Nearest: ['F2', 'F3', 'F5']
Prediction: Apple
Minkowski
Nearest: ['F2', 'F3', 'F5']
Prediction: Apple
Hamming
Nearest: ['F1', 'F2', 'F5']
Prediction: Apple
```



**Question 5: House Price Category Prediction**

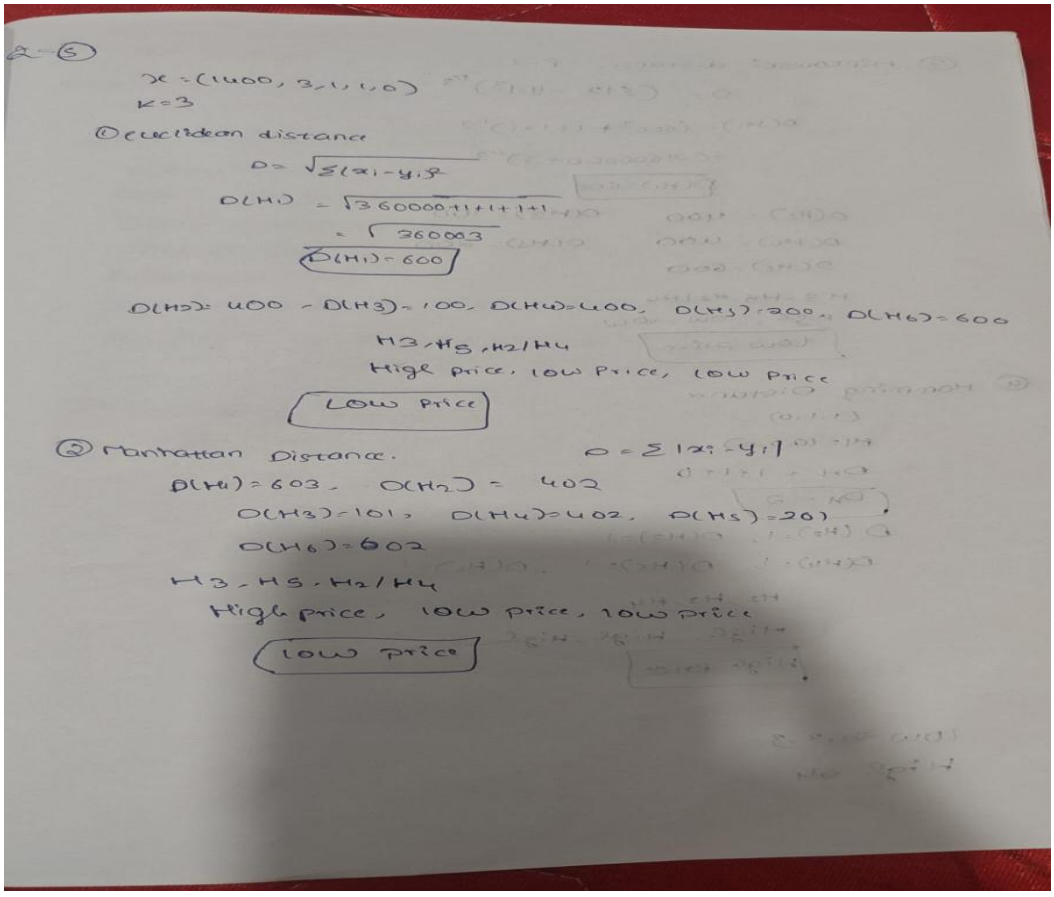
A real estate company wants to classify houses as Low Price or High Price.

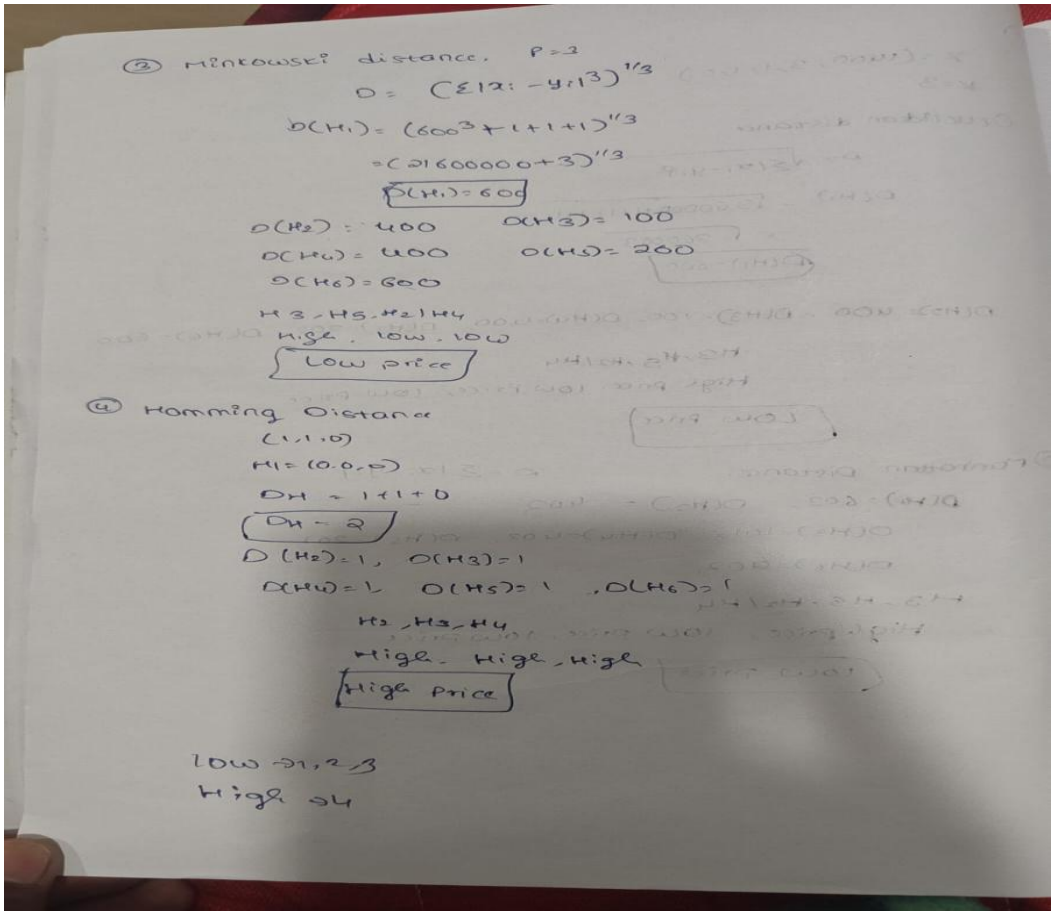
| House | Area in sq.ft | Bedrooms | Parking Available | Near Main Road | Garden Available | Category |
|---|---|---|---|---|---|---|
| H1 | 800 | 2 | 0 | 0 | 0 | Low Price |
| H2 | 1000 | 2 | 1 | 0 | 0 | Low Price |
| H3 | 1500 | 3 | 1 | 1 | 1 | High Price |
| H4 | 1800 | 4 | 1 | 1 | 1 | High Price |
| H5 | 1200 | 3 | 0 | 1 | 0 | Low Price |
| H6 | 2000 | 4 | 1 | 1 | 1 | High Price |

A new house has:

| Area | Bedrooms | Parking Available | Near Main Road | Garden Available |
|---|---|---|---|---|
| 1400 | 3 | 1 | 1 | 0 |

Using KNN with K = 3, classify the house as Low Price or High Price using Euclidean, Manhattan, Minkowski, and Hamming distance.






**Code:**

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

from google.colab import files


uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)


features = [
```

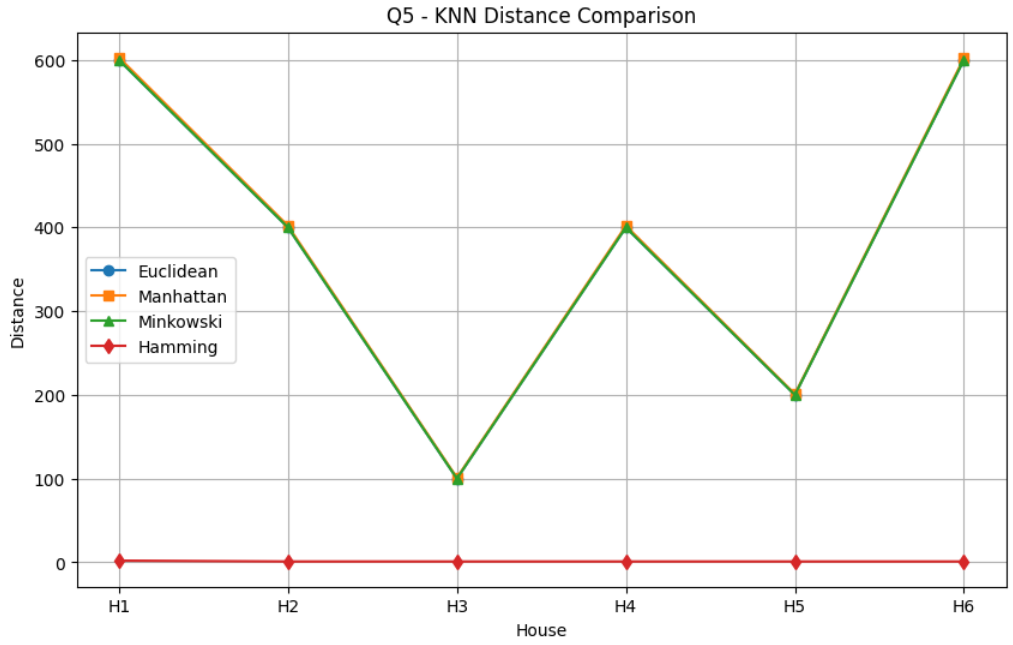
```
    "Area",
    "Bedrooms",
    "Parking_Available",
    "Near_Main_Road",
    "Garden_Available"
]

new_sample = np.array([1400, 3, 1, 1, 0])

X = df[features].astype(float).values

euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)

binary_features = [
    "Parking_Available",
    "Near_Main_Road",
    "Garden_Available"
]

hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 1, 0]),
    axis=1
)
```

```
result = pd.DataFrame({
    "House": df["House"],
    "Category": df["Category"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})

display(result)

for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Category"]).most_common(1)[0][0]
    print(method)
    print("Nearest:", list(nearest["House"]))
    print("Prediction:", prediction)

plt.figure(figsize=(10, 6))
plt.plot(result["House"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["House"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["House"], result["Minkowski"], marker="^", label="Minkowski")
plt.plot(result["House"], result["Hamming"], marker="d", label="Hamming")
plt.xlabel("House")
```

```
plt.ylabel("Distance")

plt.title("Q5 - KNN Distance Comparison")

plt.legend()

plt.grid(True)

plt.show()
```

**Output:**

```
Euclidean
Nearest: ['H3', 'H5', 'H2']
Prediction: Low Price
Manhattan
Nearest: ['H3', 'H5', 'H2']
Prediction: Low Price
Minkowski
Nearest: ['H3', 'H5', 'H2']
Prediction: Low Price
Hamming
Nearest: ['H2', 'H3', 'H4']
Prediction: High Price
```



**Question 6: Loan Approval Prediction**

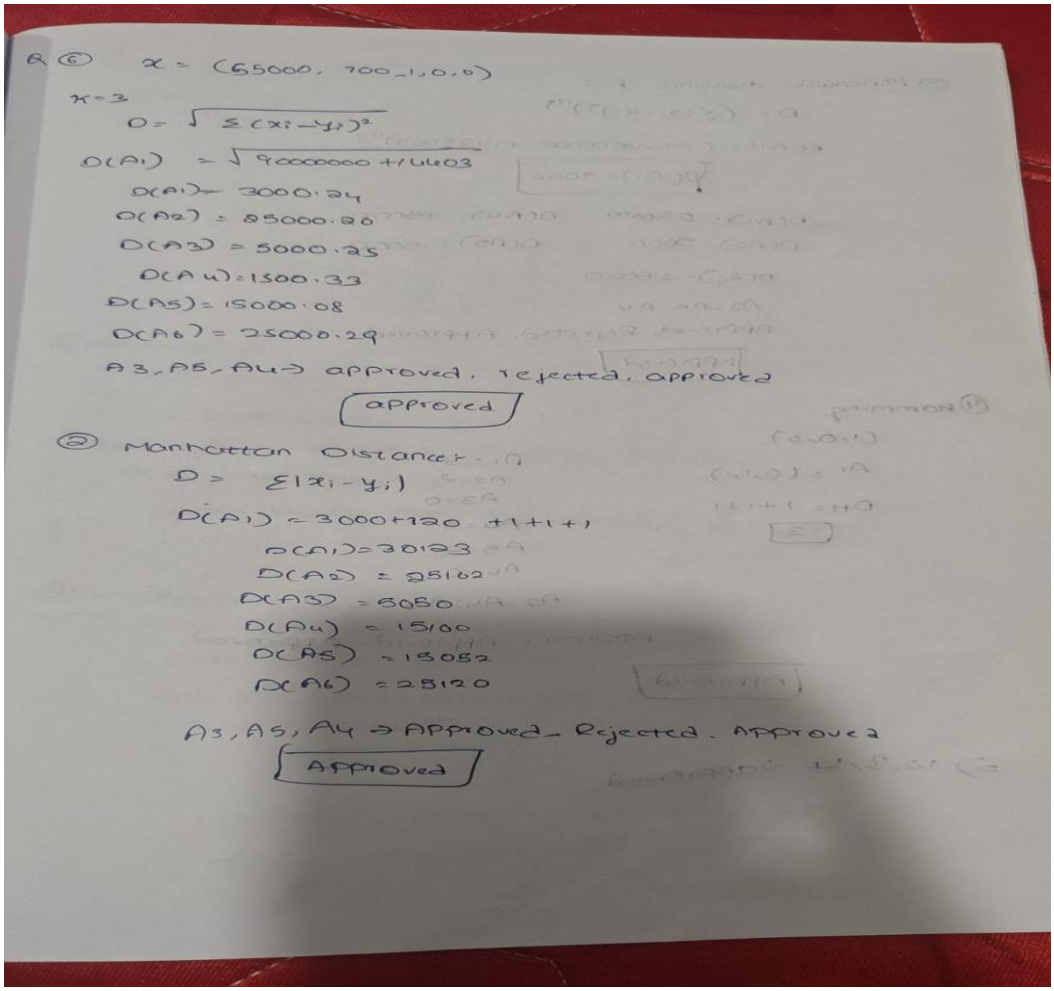
A bank wants to predict whether a loan application should be Approved or Rejected.

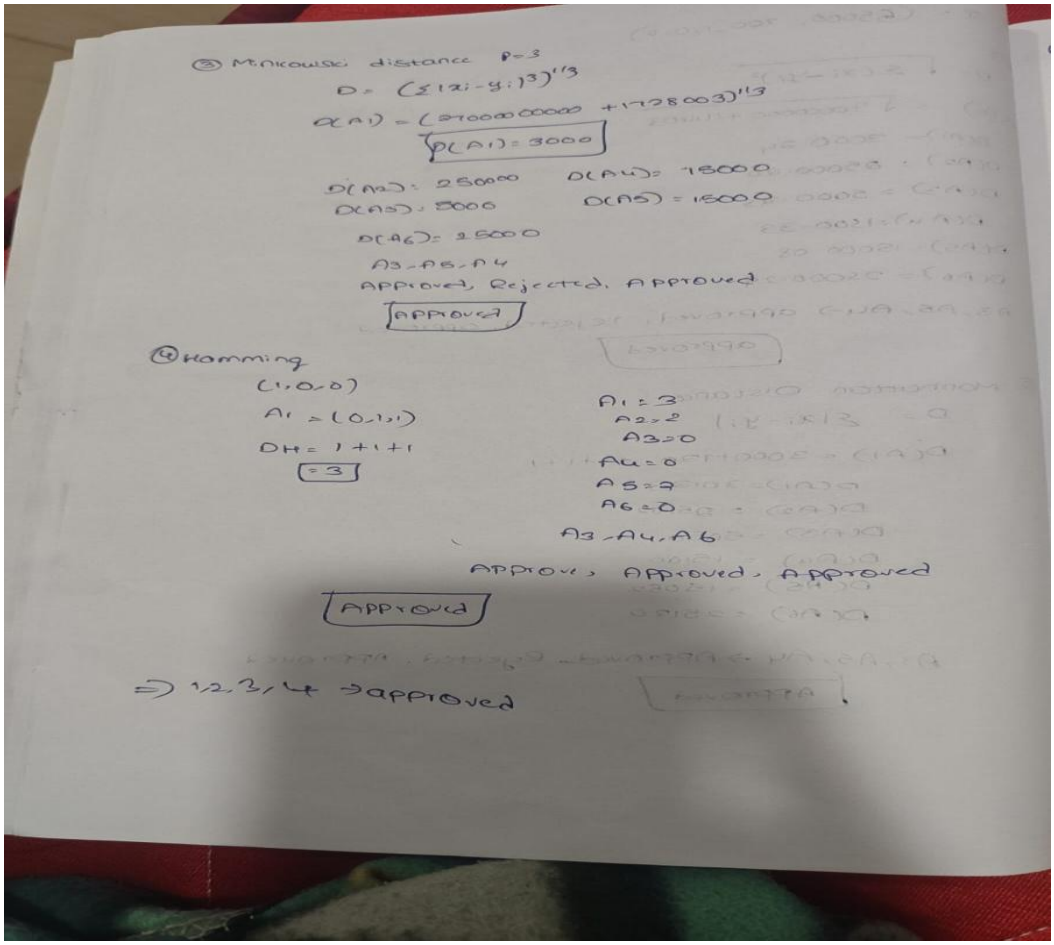
| Applicant | Monthly Income | Credit Score | Govt Job | Existing Loan | Past Default | Class |
|---|---|---|---|---|---|---|
| A1 | 25000 | 580 | 0 | 1 | 1 | Rejected |
| A2 | 30000 | 600 | 0 | 1 | 0 | Rejected |
| A3 | 60000 | 750 | 1 | 0 | 0 | Approved |
| A4 | 70000 | 800 | 1 | 0 | 0 | Approved |
| A5 | 40000 | 650 | 0 | 1 | 0 | Rejected |
| A6 | 80000 | 820 | 1 | 0 | 0 | Approved |

A new applicant has:

| Monthly Income | Credit Score | Govt Job | Existing Loan | Past Default |
|---|---|---|---|---|
| 55000 | 700 | 1 | 0 | 0 |

Using KNN with K = 3, classify the loan application as Approved or Rejected using all four distance measures.






**Code:**

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

from google.colab import files


uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)


features = [
```

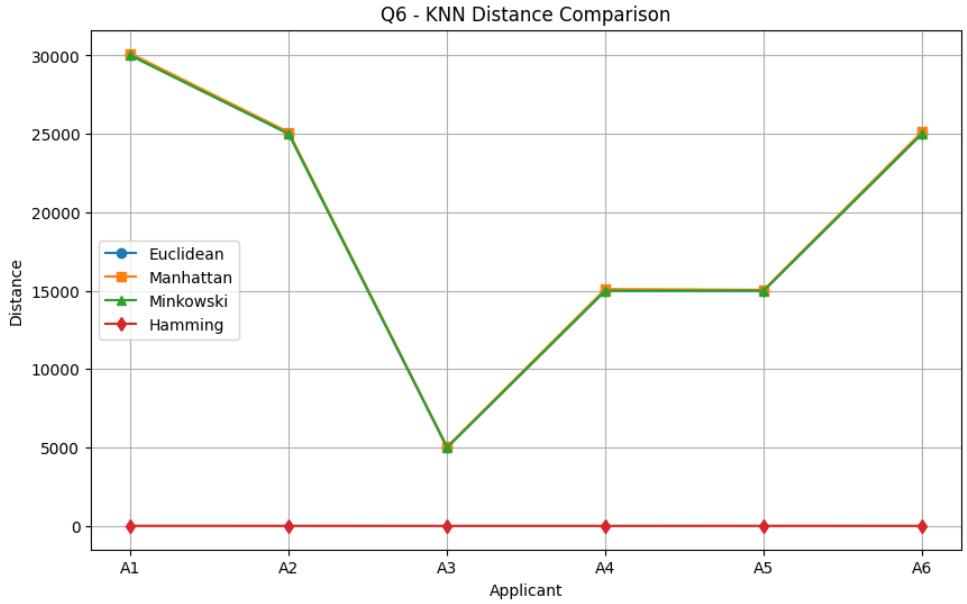
```
    "Monthly_Income",
    "Credit_Score",
    "Govt_Job",
    "Existing_Loan",
    "Past_Default"
]

new_sample = np.array([55000, 700, 1, 0, 0])

X = df[features].astype(float).values

euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)

binary_features = [
    "Govt_Job",
    "Existing_Loan",
    "Past_Default"
]

hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 0, 0]),
    axis=1
)
```

```
result = pd.DataFrame({
    "Applicant": df["Applicant"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})

display(result)

for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
    print(method)
    print("Nearest:", list(nearest["Applicant"]))
    print("Prediction:", prediction)

plt.figure(figsize=(10, 6))
plt.plot(result["Applicant"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["Applicant"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["Applicant"], result["Minkowski"], marker="^", label="Minkowski")
plt.plot(result["Applicant"], result["Hamming"], marker="d", label="Hamming")
plt.xlabel("Applicant")
```

```
plt.ylabel("Distance")

plt.title("Q6 - KNN Distance Comparison")

plt.legend()

plt.grid(True)

plt.show()
```

**Output:**

```
Euclidean
Nearest: ['A3', 'A5', 'A4']
Prediction: Approved
Manhattan
Nearest: ['A3', 'A5', 'A4']
Prediction: Approved
Minkowski
Nearest: ['A3', 'A5', 'A4']
Prediction: Approved
Hamming
Nearest: ['A3', 'A4', 'A6']
Prediction: Approved
```



**Question 7: Movie Preference Classification**

A movie platform wants to classify a user as an Action Lover or Comedy Lover.

| User | Action Rating | Comedy Rating | Watched Superhero Movie | Watched Stand-up Show | Watched Sitcom | Class |
|---|---|---|---|---|---|---|
| U1 | 5 | 1 | 1 | 0 | 0 | Action Lover |
| U2 | 4 | 2 | 1 | 0 | 0 | Action Lover |
| U3 | 1 | 5 | 0 | 1 | 1 | Comedy Lover |
| U4 | 2 | 4 | 0 | 1 | 1 | Comedy Lover |
| U5 | 5 | 2 | 1 | 0 | 0 | Action Lover |
| U6 | 1 | 4 | 0 | 1 | 1 | Comedy Lover |

A new user has:

| Action Rating | Comedy Rating | Watched Superhero Movie | Watched Stand-up Show | Watched Sitcom |
|---|---|---|---|---|
| 3 | 3 | 1 | 1 | 0 |

Using KNN with K = 3, classify the new user as Action Lover or Comedy Lover using Euclidean, Manhattan, Minkowski, and Hamming distance.

(Q7) $x = (3, 3, -1, -1, 0)$

① euclidean

$$D = \sqrt{\sum (x_i - y_i)^2}$$

$D(u_1) = \sqrt{4+4+0+1+0}$

$= \sqrt{9} = 3$

$D(u_2) = 1.73$, $D(u_3) = 3.16$, $D(u_4) = 2.00$, $D(u_5) = 2.45$

$D(u_6) = 2.65$

$u_2, u_4, u_5$ → action, comedy, action

**action**

② Manhattan

$$D = \sum |x_i - y_i|$$

$= 2+2+0+1+0$

$D(u_1) = 5$, $D(u_2) = 3$, $D(u_3) = 6$, $D(u_4) = 4$,

$D(u_5) = 4$, $D(u_6) = 5$

$u_2, u_4, u_5$ → action, comedy, action

**action**

③ Minkowski

$p = 3$

$$D = \left(\sum |x_i - y_i|^3\right)^{1/3}$$

$D(u_1) = (8+8+0+1+0)^{1/3}$

$= 17^{1/3}$

$D(u_1) = 2.57$

$D(u_2) = 1.44$, $D(u_3) = 2.62$, $D(u_4) = 1.59$

$D(u_5) = 2.15$, $D(u_6) = 2.22$

$u_2, u_4, u_5$ → action, comedy, action

**action**

④ Hamming

$(1, 1, 0)$

$u_1 = 1$, $u_2 = 1$, $u_3 = 2$, $u_4 = 2$, $u_5 = 1$, $u_6 = 2$

$u_1, u_2, u_5$

action, action, action

**action**

ction → euclidean, Manhattan, Minkowski, Hamming

**Code:**

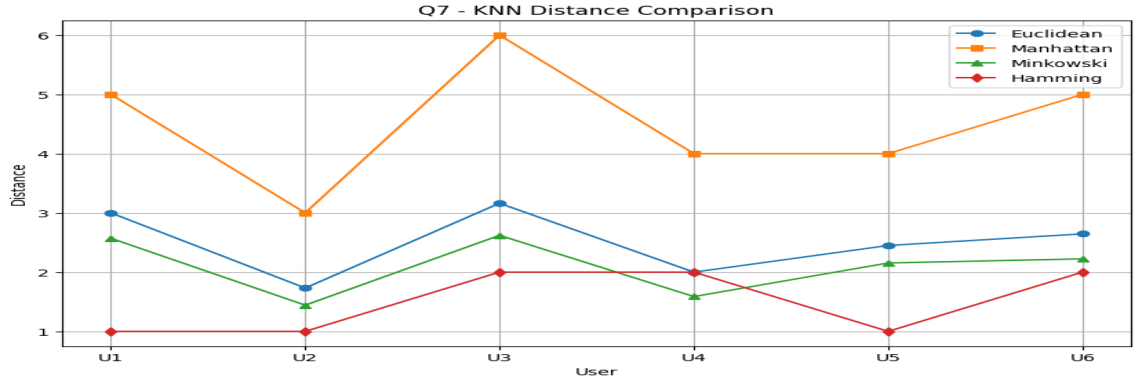
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from google.colab import files

uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

features = [
    "Action_Rating",
    "Comedy_Rating",
    "Watched_Superhero",
    "Watched_Standup",
    "Watched_Sitcom"
]

new_sample = np.array([3, 3, 1, 1, 0])

X = df[features].astype(float).values

euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
```

```
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)


binary_features = [
    "Watched_Superhero",
    "Watched_Standup",
    "Watched_Sitcom"
]


hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 1, 0]),
    axis=1
)


result = pd.DataFrame({
    "User": df["User"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})
display(result)
for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
```

```
    print(method)
    print("Nearest:", list(nearest["User"]))
    print("Prediction:", prediction)
plt.figure(figsize=(10, 6))
plt.plot(result["User"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["User"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["User"], result["Minkowski"], marker="^", label="Minkowski")
plt.plot(result["User"], result["Hamming"], marker="d", label="Hamming")
plt.xlabel("User")
plt.ylabel("Distance")
plt.title("Q7 - KNN Distance Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

**Output:**

```
Euclidean
Nearest: ['U2', 'U4', 'U5']
Prediction: Action Lover
Manhattan
Nearest: ['U2', 'U4', 'U5']
Prediction: Action Lover
Minkowski
Nearest: ['U2', 'U4', 'U5']
Prediction: Action Lover
Hamming
Nearest: ['U1', 'U2', 'U5']
Prediction: Action Lover
```



**Question 8: Crop Disease Detection**

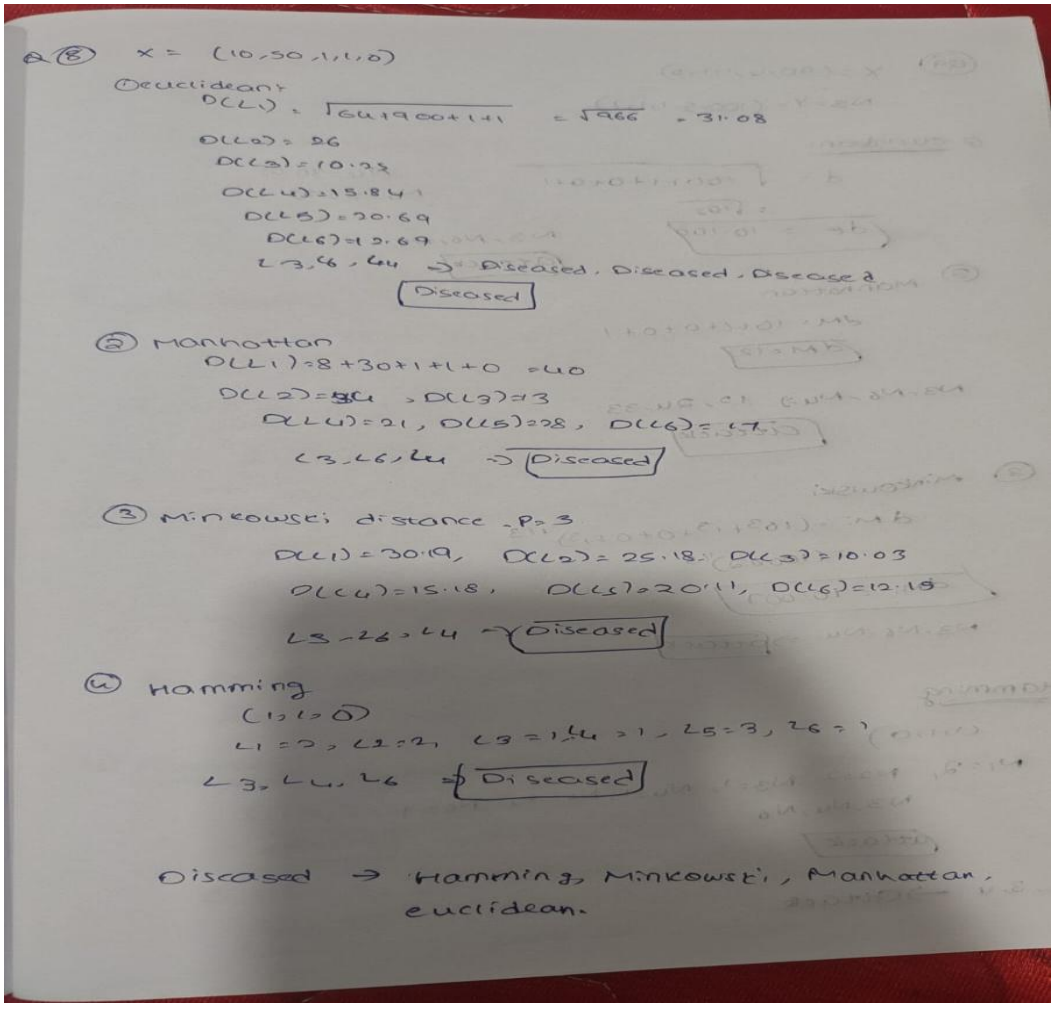
An agriculture research center wants to classify a plant as Healthy or Diseased.

| Plant | Leaf Spot Count | Leaf Color Index | Yellow Leaves | White Fungus | Wilting | Class |
|---|---|---|---|---|---|---|
| L1 | 2 | 80 | 0 | 0 | 0 | Healthy |
| L2 | 3 | 75 | 0 | 0 | 0 | Healthy |
| L3 | 12 | 40 | 1 | 1 | 1 | Diseased |
| L4 | 15 | 35 | 1 | 1 | 1 | Diseased |
| L5 | 5 | 70 | 0 | 0 | 1 | Healthy |
| L6 | 14 | 38 | 1 | 1 | 1 | Diseased |

A new plant has:

| Leaf Spot Count | Leaf Color Index | Yellow Leaves | White Fungus | Wilting |
|---|---|---|---|---|
| 10 | 50 | 1 | 1 | 0 |

Using KNN with K = 3, classify the plant as Healthy or Diseased using all four distance measures.



**Code:**

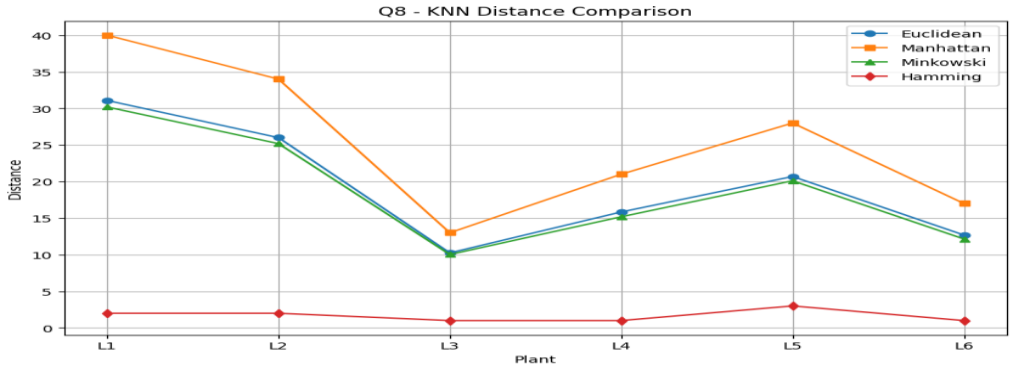
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from google.colab import files

uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

features = [
    "Leaf_Spot_Count",
    "Leaf_Color_Index",
    "Yellow_Leaves",
    "White_Fungus",
    "Wilting"
]

new_sample = np.array([10, 50, 1, 1, 0])

X = df[features].astype(float).values

euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
```

```
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)

binary_features = [
    "Yellow_Leaves",
    "White_Fungus",
    "Wilting"
]

hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 1, 0]),
    axis=1
)

result = pd.DataFrame({
    "Plant": df["Plant"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})

display(result)

for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
```

```
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
    print(method)
    print("Nearest:", list(nearest["Plant"]))
    print("Prediction:", prediction)

plt.figure(figsize=(10, 6))
plt.plot(result["Plant"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["Plant"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["Plant"], result["Minkowski"], marker="^", label="Minkowski")
plt.plot(result["Plant"], result["Hamming"], marker="d", label="Hamming")
plt.xlabel("Plant")
plt.ylabel("Distance")
plt.title("Q8 - KNN Distance Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

**Output:**

```
Euclidean
Nearest: ['L3', 'L6', 'L4']
Prediction: Diseased
Manhattan
Nearest: ['L3', 'L6', 'L4']
Prediction: Diseased
Minkowski
Nearest: ['L3', 'L6', 'L4']
Prediction: Diseased
Hamming
Nearest: ['L3', 'L4', 'L6']
Prediction: Diseased
```




**Question 9: Network Intrusion Detection**

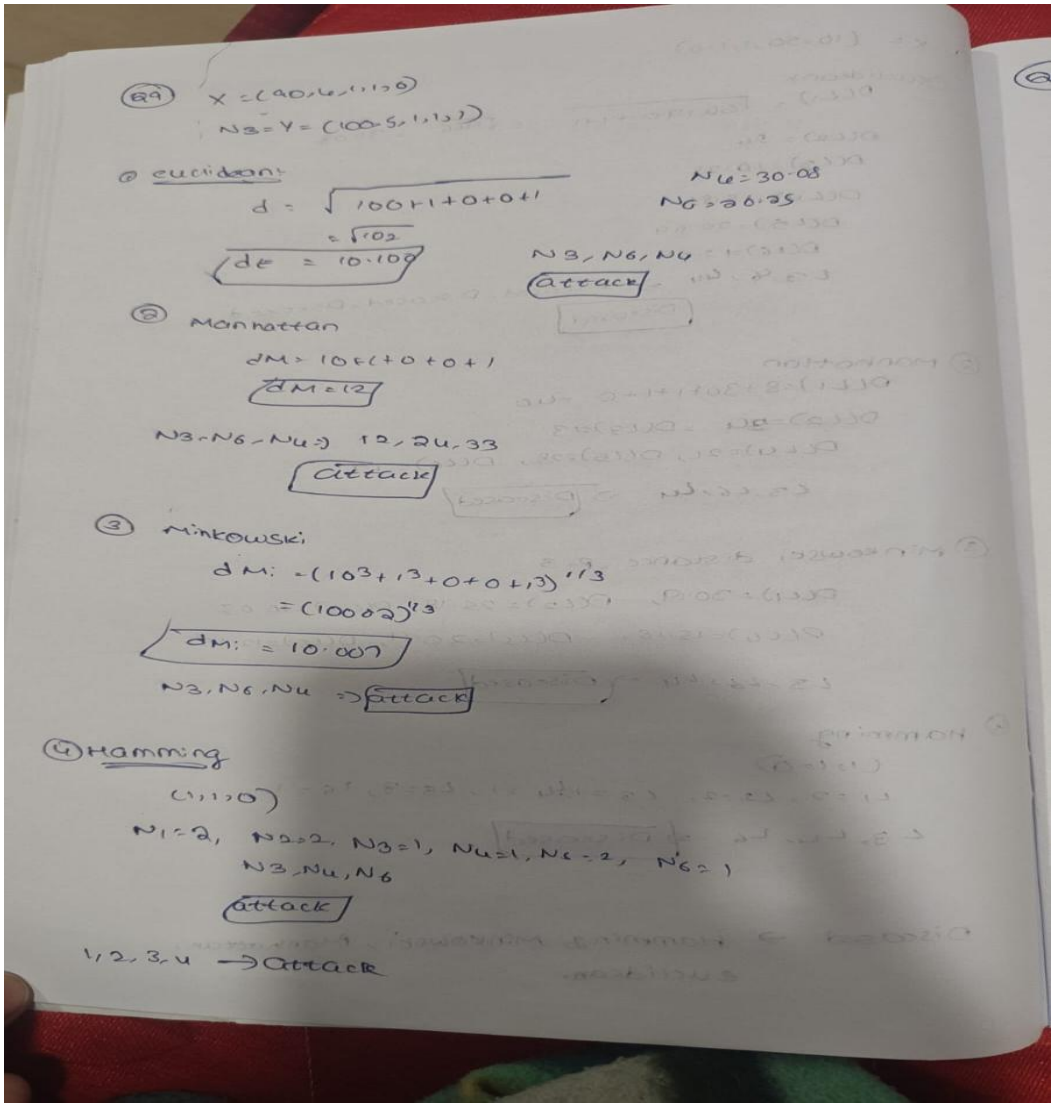
A cybersecurity system wants to classify network activity as Normal or Attack.

| Record | Packets per Second | Failed Login Attempts | Unknown Port Used | Admin Access Tried | Blacklisted IP | Class |
|---|---|---|---|---|---|---|
| N1 | 20 | 0 | 0 | 0 | 0 | Normal |
| N2 | 30 | 1 | 0 | 0 | 0 | Normal |
| N3 | 100 | 5 | 1 | 1 | 1 | Attack |
| N4 | 120 | 6 | 1 | 1 | 1 | Attack |
| N5 | 40 | 1 | 0 | 0 | 0 | Normal |
| N6 | 110 | 7 | 1 | 1 | 1 | Attack |

A new network record has:

| Packets per Second | Failed Login Attempts | Unknown Port Used | Admin Access Tried | Blacklisted IP |
|---|---|---|---|---|
| 90 | 4 | 1 | 1 | 0 |

Using KNN with K = 3, classify the network activity as Normal or Attack using Euclidean, Manhattan, Minkowski, and Hamming distance.




**Code:**

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

from google.colab import files


uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)
```

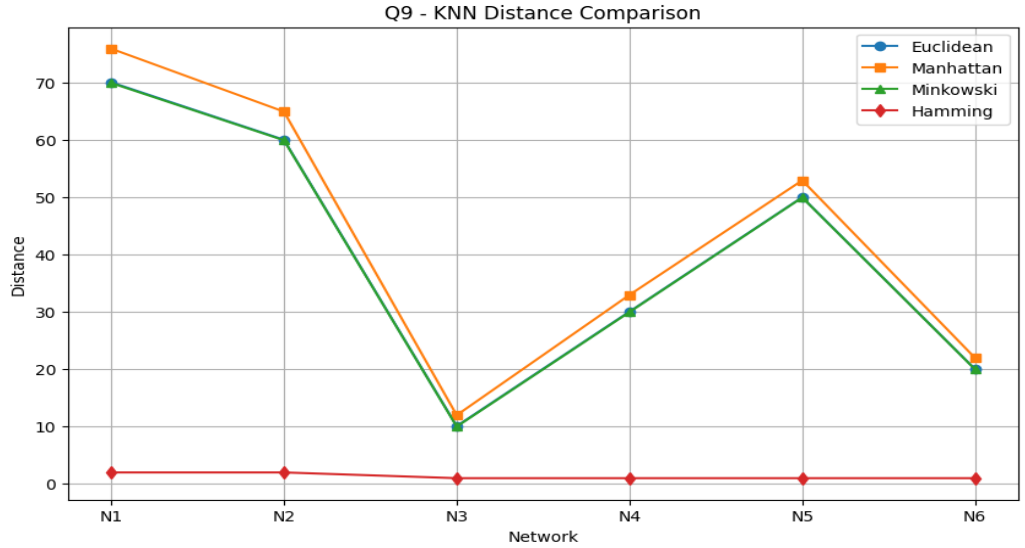
```
features = [
    "Packet_Rate",
    "Connection_Count",
    "Unknown_Port",
    "Admin_Login",
    "Blacklisted_IP"
]

new_sample = np.array([90, 4, 1, 1, 0])

X = df[features].astype(float).values

euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)

binary_features = [
    "Unknown_Port",
    "Admin_Login",
    "Blacklisted_IP"
]

hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 1, 0]),
```

```
    axis=1
)

result = pd.DataFrame({
    "Network": df["Network"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})

display(result)

for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
    print(method)
    print("Nearest:", list(nearest["Network"]))
    print("Prediction:", prediction)

plt.figure(figsize=(10, 6))
plt.plot(result["Network"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["Network"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["Network"], result["Minkowski"], marker="^", label="Minkowski")
```

```
plt.plot(result["Network"], result["Hamming"], marker="d", label="Hamming")

plt.xlabel("Network")

plt.ylabel("Distance")

plt.title("Q9 - KNN Distance Comparison")

plt.legend()

plt.grid(True)

plt.show()
```

**Output:**

```
Euclidean
Nearest: ['N3', 'N6', 'N4']
Prediction: Attack
Manhattan
Nearest: ['N3', 'N6', 'N4']
Prediction: Attack
Minkowski
Nearest: ['N3', 'N6', 'N4']
Prediction: Attack
Hamming
Nearest: ['N3', 'N4', 'N5']
Prediction: Attack
```



**Question 10: Employee Attrition Prediction**

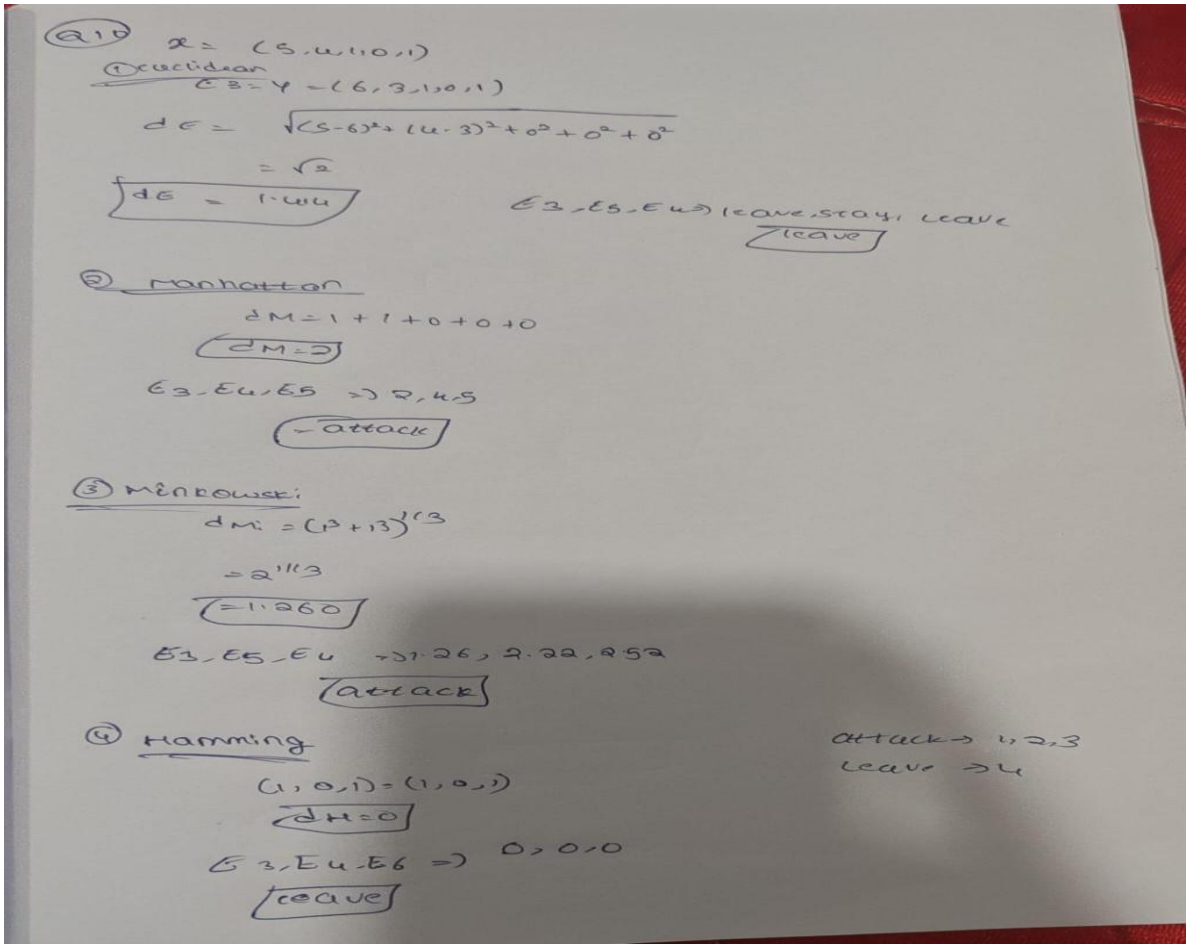
A company wants to predict whether an employee is likely to Stay or Leave.

| Employee | Experience in Years | Satisfaction Score | Overtime | Promotion Received | Workplace Conflict | Class |
|---|---|---|---|---|---|---|
| E1 | 2 | 8 | 0 | 1 | 0 | Stay |
| E2 | 3 | 7 | 0 | 1 | 0 | Stay |
| E3 | 6 | 3 | 1 | 0 | 1 | Leave |
| E4 | 7 | 2 | 1 | 0 | 1 | Leave |
| E5 | 4 | 6 | 0 | 0 | 0 | Stay |
| E6 | 8 | 2 | 1 | 0 | 1 | Leave |

A new employee has:

| Experience in Years | Satisfaction Score | Overtime | Promotion Received | Workplace Conflict |
|---|---|---|---|---|
| 5 | 4 | 1 | 0 | 1 |

Using KNN with K = 3, classify the employee as Stay or Leave using all four distance measures.



**Code:**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from google.colab import files

uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

features = [
    "Years_Experience",
    "Job_Satisfaction",
    "Overtime",
    "Promotion",
    "Workplace_Conflict"
]

new_sample = np.array([5, 4, 1, 0, 1])

X = df[features].astype(float).values

euclidean = np.sqrt(np.sum((X - new_sample) ** 2, axis=1))
manhattan = np.sum(np.abs(X - new_sample), axis=1)
```

```
minkowski = np.sum(np.abs(X - new_sample) ** 3, axis=1) ** (1/3)


binary_features = [
    "Overtime",
    "Promotion",
    "Workplace_Conflict"
]


hamming = np.sum(
    df[binary_features].astype(int).values != np.array([1, 0, 1]),
    axis=1
)


result = pd.DataFrame({
    "Employee": df["Employee"],
    "Class": df["Class"],
    "Euclidean": euclidean,
    "Manhattan": manhattan,
    "Minkowski": minkowski,
    "Hamming": hamming
})
display(result)
for method in ["Euclidean", "Manhattan", "Minkowski", "Hamming"]:
    nearest = result.sort_values(method, kind="stable").head(3)
    prediction = Counter(nearest["Class"]).most_common(1)[0][0]
```

```
    print(method)
    print("Nearest:", list(nearest["Employee"]))
    print("Prediction:", prediction)
plt.figure(figsize=(10, 6))
plt.plot(result["Employee"], result["Euclidean"], marker="o", label="Euclidean")
plt.plot(result["Employee"], result["Manhattan"], marker="s", label="Manhattan")
plt.plot(result["Employee"], result["Minkowski"], marker="^", label="Minkowski")
plt.plot(result["Employee"], result["Hamming"], marker="d", label="Hamming")
plt.xlabel("Employee")
plt.ylabel("Distance")
plt.title("Q10 - KNN Distance Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

**Output:**

```
Euclidean
Nearest: ['E3', 'E4', 'E2']
Prediction: Leave
Manhattan
Nearest: ['E3', 'E4', 'E2']
Prediction: Leave
Minkowski
Nearest: ['E3', 'E4', 'E2']
Prediction: Leave
Hamming
Nearest: ['E2', 'E3', 'E4']
Prediction: Leave
```